Keeping the Why

Armature and Design Data Infrastructure for Learning Engineering

Amos Glenn*

Learning engineering has mature infrastructure for recording what learners do and almost none for recording how the designs they experience came to be. The reasoning that connects a learning need to objectives, activities, assessments and outcomes has no system of record. Within a project, the relations between design artifacts live in documents and memory. And there is no shared bridge between design and learning data. This paper introduces Armature, open infrastructure for design data: a schema and an API over a versioned graph that records design artifacts, the named relations between them, and every change with its author and reason. Tools are designed to write to the graph as a byproduct of ordinary design work, and the graph returns analysis computed from its structure. The paper then checks Armature's architectural decisions against the literature on earlier attempts to record design, from executable learning design languages to design rationale systems and decision records. Those attempts mostly faded for recurring reasons, and Armature's main decisions are designed to answer them. Two conditions lie beyond what the architecture can supply: reasons good enough to use, and a professional norm that sustains the record.

Keywords: learning engineering, instructional design, design rationale, design data, alignment, version control, provenance

1. Introduction

Here is a common scenario. A university notices that students consistently struggle with one concept on a biology assessment, and it asks a learning engineer for help. The engineer's first questions are predictable. Where in the course are students supposed to develop this skill? Are the activities for that module still aligned with the objective? Does the assessment item still measure what it was written to measure?

Answering those questions means digging through curriculum committee minutes, syllabi revised by successive instructors, answer keys maintained by technologists, and emails that someone remembers but cannot find. Even with the best paper trails, the information is fragmented. What is missing is the connections between the pieces: which need produced this objective, which activities were meant to practice it, which items were meant to measure it, and how and why each of those has changed since.

Learning engineering has built sophisticated infrastructure for other parts of its process. Learning management systems, xAPI statements, learning record stores, and platforms such as DataShop capture what learners did and how they performed. A few systems, notably OLI Torus, also hold links to design. There is no comparable system of record for design and the reasoning that connects a learning problem to objectives, objectives to content and assessment, and all of it to outcomes.

This paper makes the case for building one. It introduces Armature, open infrastructure for what it calls design data: the artifacts of instructional design, the named relations between them, and the history of how both changed and why. It describes Armature as it is built today, indicating when something is only planned, and then checks Armature's main architectural decisions against what the literature records about earlier attempts to capture design: executable learning design languages, design rationale systems, metadata standards, and the decision records and version histories that software teams keep.

*Send correspondence to: Amos Glenn, amos@amosglenn.com.

The need is timely as generative AI improves its ability to draft and tag content. In one study, experts preferred GPT-4′s knowledge-component labels to the human-written ones about two-thirds of the time (Moore et al., 2024). In another, most of the design rationale a language model produced did not match what experts had actually argued, yet most of its additional arguments were judged helpful (Zhou et al., 2026). Good machine output will be accepted, so it must stay attributable and reviewable. The design record matters more now than it did when every artifact had a human author.

Three limitations should frame this discussion. The paper reports no user data; Armature is in early development, with one demonstration course and one plugin whose connection to the hub is designed but not yet built. It is not a systematic review: the literature is selected to test Armature's decisions, and the strongest counter-evidence found is reported alongside the support. And it does not claim that Armature improves design or learning. It claims that such an architecture is needed and that Armature's architecture is designed to address the main causes of earlier attempts' failures, and it names the conditions for success that infrastructure cannot supply.

Section 2 sets out the problem, and Section 3 defines design data and the partial solutions that already exist. Section 4 describes Armature and what a design graph makes possible. Section 5 draws lessons from earlier attempts to record design and shows how Armature's architecture answers them. Section 6 takes up the strongest objections, and Section 7 concludes.

2. The problem: design has no system of record

In software, a system of record is the authoritative store for a kind of information. A student information system is the system of record for enrollment; a learning management system is the system of record for grades. Instructional design has no system of record for its process. The absence shows up as two gaps.

The internal gap

Within a design project, every tool manages its own artifacts competently. Survey platforms hold needs-assessment data, authoring tools hold content, learning management systems hold quizzes, and spreadsheets hold curriculum maps. Nothing was built to model the relations between those artifacts across the design process, or to keep the reasons behind them: that this objective was written in response to that need, that these three items are meant to measure it, that this activity was placed third for a reason. Those relations and reasons live in documents, in conversations and in the heads of the people who made them. When the people leave and the course is revised, the relations go with them.

This gap is well known. The authors of the *Learning Engineering Toolkit*'s chapter on design tools call keeping track of the reasons for decisions "a significant challenge" (Schatz et al., 2023, p. 290), and Thai et al. call tracking design decisions "no small undertaking" (2023, p. 96). Totino and Kessler note that "the practical work of how such design decisions can be recorded and maintained for research and practical purposes is drastically underreported in the literature" (2024, p. 277), and Sandoval points directly to the lack of design research that reports how data from one enactment justified the revisions that followed (Sandoval, 2014, p. 32). Jonassen is even more critical: "Ask any instructional designer to justify all of the design decisions that were made, and the designer will not be able to provide empirical or rational justifications for many of the decisions" (2008, pp. 23–24). Boling calls instructional design "a notable exception" among all design fields for lacking an accessible body of precedent (2010, p. 6).

The most rigorous attempt within learning engineering to close this gap is the Learning Engineering Evidence and Decision (LEED) tracker, a deliberately "low-tech, low-overhead, and shareable" table in which teams record each decision, its source of influence, and its justification (Totino & Kessler, 2024, p. 278). Its authors' study of three years of use is the best evidence available on how decision capture works in learning engineering. Eighteen projects logged 283 decisions (Totino & Kessler, 2025, pp. 819–828). Capture was feasible and served communication and hand-off. It was also uneven, as

several projects logged five or fewer decisions, and one external project stopped recording after its first five. And few decisions were linked to evidence; only 14 of the 283 decisions cited data, all of them in internal projects. LEED showed that a lightweight table can record that a decision was made and why. It also showed that such capture lapses without someone sustaining it, and that a table cannot connect each decision to the other decisions and the data that might later test it.

The external gap

There is no shared bridge between design data and learning data. Learning engineering's infrastructure is concentrated on the learner side of its own process. In the Learning Engineering Toolkit, the chapter on Learning Engineering Uses Data (Part 1): Instrumentation defines instrumentation as "designing, developing, and implementing the data collection used within the learning solution" (Czerwinski et al., 2023, p. 153). Similarly, the ICICLE model of the learning engineering process places "instrument the data" as one component of "create the solution," while "collect data" is part of the "Implement the designed solution" phase (IEEE ICICLE, n.d., "What Does the Learning Engineering Process Look Like?" section). In both, what gets instrumented is data about learners and their performance; neither names the design itself, or the decisions behind it, as something to record. Figure 1 places today's tools on the learning engineering process. The learning analytics community has noticed the same absence from its side: a systematic review of analytics for learning design found that the studies "failed to report course meta-data, such as data regarding the course structure, resources or teaching concepts" (Mangaroska & Giannakos, 2019, p. 528).

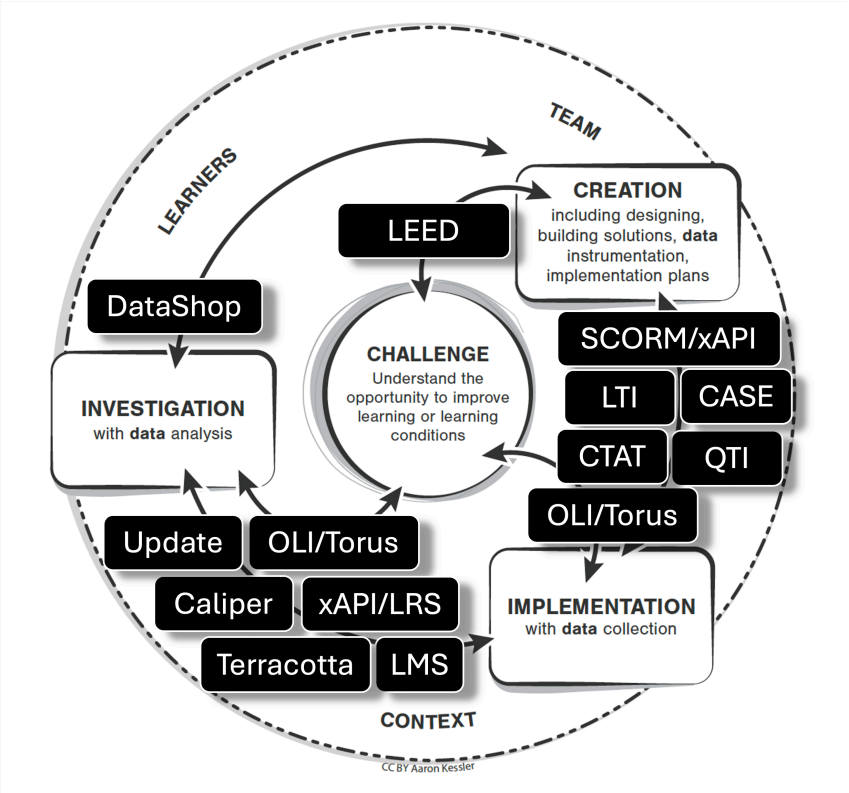


Figure 1. Where learning engineering's data infrastructure sits. Frameworks for using learning data and design data (black labels) are mapped onto the learning engineering process (CC-BY Aaron Kessler, adapted from Kessler et al., 2023, p. 31). Established frameworks cluster around implementation and investigation. The space between investigation and creation, where design data belongs, is empty.

The cost shows when results need explaining. Kessler et al. (2023) describe a project in which three educators implemented the same tutoring system: "Each educator operationalized and enacted the system in a different way, leading to mixed results. Much of the complexity came from differences in the specific learning goals that each of the educators hoped to address" (p. 41). Without a record of each educator's goals and of how each adapted the design, mixed results like these cannot be traced to their causes. Kessler et al. suggest that the problem belongs on the field's agenda: "In fact, understanding how systems are implemented and the impacts of such work could be a separate learning engineering challenge that must be addressed" (p. 41).

Some systems do link outcome data back to specific design artifacts. Aleven et al. (2026, pp. 483, 488, 494, 498) describe several examples of design-loop adaptations that leverage DataShop's mapping of learning activities to knowledge components. The Open University mapped 189 module designs by hand and linked 151 of them to outcome data (Rienties & Toetenel, 2016). These links are real, but each was built for one purpose: a mapping of items to skills inside one research platform, or a retrospective mapping made by hand. Neither records the design process that produced the artifacts being linked.

Where no such link exists, learner-data standards strain when asked to carry design meaning. One national project using xAPI recorded the competence objective an activity addressed by registering "competence objective" as a kind of activity, which its contributors described as "a minor 'hack'" adopted because it made querying easier (Samuelsen et al., 2021, p. 16).

The success cases show the power of learner data to improve learning activities, but they leave out the design process for turning those outcomes into better learning. What is missing is not every kind of design data, but a shared, continuously written record of design that can be joined to learning data.

Why it matters for learning engineering

Learning engineering studies rigorously what works: which learning experiences produce which outcomes. What it cannot yet study is which design practices produce things that work, because the design process is not recorded in a form that can be compared across projects. Questions like these are currently unanswerable at scale:

- Are there design patterns that predict poor learning outcomes?
- Do certain approaches to needs assessment produce more durable learning objectives?
- When an AI system proposes a design artifact and a person accepts it without modification, is it as reliable as one a person wrote?

3. Design data

Design data is a structured, queryable, versioned record of the instructional design process that produces a learning experience. It has three parts: artifacts, relations, and history.

Design artifacts are the concrete products of design: needs-assessment findings, learning objectives, activities, assessment items, modules. They are familiar; every designer already works with them.

Design relations are meaningfully named connections between artifacts, each expressing a design decision: *this need produced that objective*, *this item measures that objective*, *this objective is a prerequisite for that one, for this reason.* Relations are as important to a design as its artifacts. In practice today they are implicit. A designer knows which activity targets which objective, and colleagues may infer it from context or possibly reconstruct it from the fragmented paper trail. But when the designer leaves and the course is revised, the connection is gone unless someone recorded it in a form that can be queried. Figure 2 shows a simplified set of artifacts and relations.

Design history is the record of how artifacts and relations changed over time, by whom, and why. It answers the questions that matter: why was it built this way, and how did it get here?

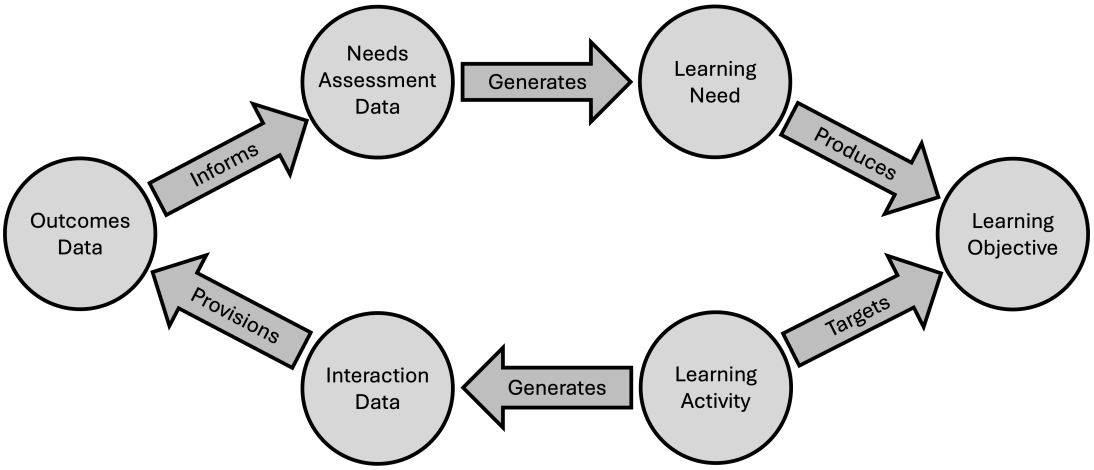


Figure 2. A simplified design graph. Each circle is an artifact and each arrow a relation with a meaningful name. The names are illustrative; Armature's schema has its own. Assessment items, omitted here, sit beside activities: they assess objectives, and their results become outcome data.

Design data differs from learning data in who produces it, when, and what it captures (Table 1). Each is valuable alone. Learning data says what happened in a learning experience; design data says how and why the experience was built. Joined, each makes the other more useful: an outcome becomes evidence about the design that produced it, and a design decision becomes something outcomes can test.

Relations are claims

A defining property of design relations is that they are judgments, not objective facts. A study of alignment judgments shows this clearly. Twenty experts mapped test items to mathematics competency standards, and their agreement was $\kappa = .55–.58$ on specific topics and .71–.74 on broad categories (Herman et al., 2005, pp. 9, 21, 28–29). Webb, whose method is among the most widely used for judging alignment, observes that "the existence of such issues reinforces that alignment judgments have an element of subjectivity" (2007, p. 7). Learning engineering's own methods treat these relations the same way. Expert-built knowledge component models start as "best guesses" (Cen et al., 2006, p. 164) and different analysts produce different ones (Stamper & Koedinger, 2011, p. 353). In design-based research, "each arrow in a conjecture map specifies a relation open to empirical refinement" (Sandoval, 2014, p. 27).

So a design relation should be recorded as an attributed, dated, revisable claim: someone asserted, at a particular time and for a stated reason, that this item measures that objective. There is precedent for this. DataShop keeps several competing knowledge component models for the same dataset (Koedinger et al., 2010). Nanopublications separate an assertion from its provenance (Groth et al., 2010), and the CaSS competency system stores judgments as assertions carrying their source, evidence, and confidence (CaSS Project, n.d.; Robson et al., 2022, p. 829).

Table 1. Design data and learning data.

	Design data	Learning data
Who records it	Designers, reviewers and their tools	Learners, through their actions
When	Accumulated across the design process	At the moment of learning
What it captures	Artifacts, relations and their history	Actions and performance

Records describe, not prescribe

Recording relations explicitly makes some familiar principles of instructional design inspectable. Constructive alignment asks that intended outcomes, teaching activities, and assessment be designed to support one another (Biggs, 1996); backward design identifies desired results and acceptable evidence before planning activities (Wiggins & McTighe, 2005); needs assessment defines a need as a gap between current and desired outcomes (Kaufman, 1972); and the revised taxonomy gives objectives and items a shared set of cognitive-process categories (Anderson & Krathwohl, 2001). A design graph can show whether a course's relations satisfy these principles. It does not require a designer to follow any of them; it records whatever relations a design actually has. This is not a claim of perfect neutrality but an attempt to follow the Larnaca Declaration on Learning Design's principle of *pluralism*, since "any system of description will have certain biases" ("The Larnaca Declaration on Learning Design," 2016, p. 9).

What already exists

Partial answers to the problem in Section 2 exist, and the claim here is about their combination, not their absence. Table 2 compares the closest systems against five properties a system of record for design would need: it holds design *artifacts*; it records named *relations* between them; it keeps a *history* in which each change carries a reason; it is *shared*, so that many tools read and write one store; and it computes *analysis from structure* at design time.

Each of these does something Armature does not attempt. Torus delivers instruction; DataShop analyzes learner data at a depth Armature will never match; curriculum maps are already familiar to accreditors. Armature's distinction is combining artifacts and named relations in one shared store, with a reason for every change, and analysis returned to designers while they work.

4. Armature: what it is and what it makes possible

Armature is infrastructure for instructional design. The name comes from sculpture: an armature is the internal frame that supports a complex sculpture and keeps it from deforming as it grows. It does not replace the artist's tools or vision. It makes more ambitious work possible.

Infrastructure, not a tool

Concretely, Armature is two things. The first is an open *schema*: a formal definition of the kinds of records a design holds and how they connect. It currently defines 27 record types (learning needs, evidence, objectives, prerequisites, assessment items, activities, modules, design notes, findings, and others) and 12 controlled vocabularies, such as the categories of the revised Bloom taxonomy. The second is an *API*, a defined set of requests other software can make: record this new item, and here is why; give me this objective as it stood last term; which objectives in this module have too few approved assessment items?

Armature is deliberately not several things it might be mistaken for. It is not a learning management system, an object repository, a learning record store, an adaptive engine, an assessment platform, or a delivery system. It is not an authoring app, a quiz builder, or a curriculum-mapping tool. It is designer-facing, not student-facing, and it works at design time, not delivery time.

Armature aims to be infrastructure in the sense that it is "sunk into, inside of, other structures, social arrangements and technologies" and becomes visible mainly when it breaks (Star & Ruhleder, 1996, p. 113). It is something designers rarely look at directly because they use it through tools that help them do the things they do already.

Table 2. Related systems against five properties of a design record. Sources: Totino & Kessler (2024); Harden (2001); Simon Initiative, Carnegie Mellon University (2026); Open edX (2026); Hernández-Leo et al. (2018); Laurillard et al. (2018); Hayashi et al. (2009); 1EdTech Consortium (n.d.); Koedinger et al. (2010).

System	Artifacts	Named relations	History with reasons	Shared by many tools	Design-time analysis
LEED tracker	Partly (table rows)	No	Partly (justifications, no versioning)	No	No
Curriculum maps	Yes	Yes (the links are the point)	No	No	Partly
OLI Torus	Yes	One type (objectives per item part)	Revisions, no reasons	Partly (within one platform)	Partly
Open edX Learning Core	Yes	Containment	Publish logs, optional messages	Partly (plugins within one platform)	No
ILDE	Yes	Partly	No	Yes (a hub of design tools)	Not found
Learning Designer	Yes	Partly	Versions	No	Yes (activity profiles)
SMARTIES	Yes	Yes (theory-justified)	No	No	Yes (theory-aware guidance)
CASE	Competencies	Yes (associations)	Framework versions	Yes	No
DataShop	Knowledge-component models	Yes (item to skill; competing models)	Models kept side by side	Yes (research community)	After delivery, from learner data
Armature	Yes	Yes	Yes (every change)	By design (first plugin not yet connected)	Yes

A hub and its plugins

Designers will not use an API. They use tools: an assessment-review tool, a curriculum mapper, an intake form for needs analysis. In Armature's architecture, these tools are *plugins*, separate applications that read and write design data through the API, and Armature is the *hub* they share. The API enforces rules that keep design data coherent: every change is attributed to a person and given a reason, and history is kept.

The first plugin being developed alongside Armature is CoQui, a collaborative quiz item review tool. CoQui organizes reviews in rounds and records subject-matter experts' findings and judgments on items and on the parts of items. Its review workflow is built; its connection to the hub, which will send each round's revisions, reasons and findings to Armature, is designed but not yet built. It was developed not only to address a real-world challenge for instructional designers, but to test what a plugin and the hub need from each other.

The Git analogy

Armature's guiding analogy is Git, the version-control system most software is developed with. Git did not replace developers' text editors. It added a layer underneath them that recorded the history of every change. Developers moved to Git's distributed version control for practical reasons of their own workflow, such as working offline, committing locally, and branching cheaply (Muşlu et al., 2014).

The history Git promised was not a factor until the history existed. Only after Git was adopted did it start creating the history developers now use daily in their work, including turning to older history to recover the rationale behind code (Codoban et al., 2015). The analogy isn't perfect. Git history is often rewritten, and whether a commit message records *why* depends on the norms of the team writing it: across five large projects, a missing *why* was the most common defect in commit messages, at 28% on average (Tian et al., 2022, p. 2394). Still, Armature aims to sit underneath design tools in the same way, so a design's history accumulates as a byproduct of the work designers already do. The lesson for Armature is in the order of adoption: a plugin has to be worth using for its own work, as CoQui must be as a review tool, before anyone values the history it leaves. And the quality of the reasons in that history will depend on norms Armature cannot supply (lesson 5, Section 5).

How Armature works

The schema follows the design lifecycle. Evidence of a need grounds a learning need. The need generates objectives, which may require other objectives as prerequisites. Activities target objectives and items assess them. Activities and assessments belong to modules, which declare which objectives they address. When an assessment is administered, it produces a dataset from which metrics are derived, and a metric can in turn become evidence of a new need, closing the loop. The schema already has record types for datasets and metrics; a path for importing them from delivery systems is planned.

Every relation is attributed through the commit that created it, so the history says who asserted it, when and why; attestations of claims let others affirm or decline it. Any relation that carries data of its own is a record in its own right. A prerequisite carries its type (hard, soft, or corequisite) and a required rationale; the link between a need and its evidence carries the designer's confidence in that evidence. Notes and findings can be attached to any record, including a relation, so a reviewer can say why a relation is wrong, not only why an artifact is, and a designer can annotate a record their current tool cannot edit.

Rules keep the design coherent. The API enforces simple rules about data (e.g., required fields and allowed values) and complex rules about structure: a reference must point to the right kind of record; an item's placement in an assessment cannot be approved while the item itself is still a draft or in review; a finding cannot be dismissed without a reason. All rules are enforced on every write, and the API reports every violation at once rather than stopping at the first.

History is first-class. Every write is a single *commit*, recorded with the person responsible and a reason; a write without a reason is refused. *Branches* let multiple designers, subject-matter experts, and stakeholders work in parallel without affecting the shared design, and a three-way *merge* reports conflicts instead of guessing. Any read can be made against any past commit, so a module's coverage could be compared before and after a set of items were retired, for example. History is never rewritten; Armature's API has no operation that resets or squashes commits, and a branch cannot be deleted unless its work has been merged into an existing branch. A mistake is undone the way Git undoes it, with a new commit that restores the earlier state and carries its own author and reason.

People and AI agents are identified. Every change is attributed to a user. An AI agent is an ordinary user whose identifier marks it as an agent; it writes through the same API, under the same rules, and its commits carry its identifier. Proving who someone is will belong to the institution's own login system. The current implementation trusts the identity a request claims, so it is suitable only for local use; single sign-on is planned.

What a design graph makes possible

Design intelligence. Most feedback in instructional design arrives too late. Outcomes data show that students struggled with a concept only after the course was delivered. The learning engineer then works backward to diagnose what went wrong. Design intelligence, Armature's term for analyses computed from a design's structure and returned to designers while they work, inverts this; the learning analytics literature calls a similar layer *design analytics* (Hernández-Leo et al., 2019, p. 4).

When the design graph accumulates artifacts and relations as designers work, some analyses that currently require post-delivery learning data become available during the design process using design data. This means that the current design structure, and possibly the implications of a section being misaligned, becomes visible to designers as they work.

Consider what a quiz authoring tool built on the design graph can do that a conventional quiz authoring tool cannot:

- Show the designer which objectives currently lack assessment coverage
- Flag when an objective is already over-assessed, so that a new item for it may be redundant in the question bank
- Surface the needs assessment data that generated the objective being assessed, giving the designer the context of the original problem beyond the abstracted objective
- Identify when the Bloom's level of a proposed item is inconsistent with the level specified for the objective it measures

Armature provides four such analyses to any plugin today. *Coverage* reports whether a module's declared objectives are adequately assessed, counting approved items separately from items still in draft or review. *Alignment* compares the cognitive level of each item with that of the objective it assesses. *Trace* walks the design lifecycle in both directions from any record: back from an outcome through items and objectives to the evidence of need, and forward from an objective to the items and activities that address it. *Impact* shows what depends on a record before it is changed. All four are computed when they are read and never stored, so they reflect the design as it is, or as it was at any past commit.

These analyses do not make the judgments behind the relations correct. They make them visible, attributable, and open to challenge. None of this requires AI. It requires structured data.

Design intelligence is meant to provide practical value immediately. Where learning analytics tells you what happened in a learning experience, design intelligence tells you what the current design structure suggests about the resulting learning experience. Jonassen (2008, pp. 23–24) describes design as model building: as decisions are made, the model of the solution becomes more elaborated. Design intelligence reads that model while it is still being built. It provides the designer with the information and insights needed to make better design decisions in real time. A designer who can see coverage gaps while writing assessments can close them before delivery. A designer who can trace an activity back to the evidence of need that generated the objective it targets is better placed to create effective activities. These are workflow improvements that instructional designers should recognize and want. Design intelligence is what makes them possible.

Iteration becomes traceable. When outcome data reveal a problem, the learning engineer currently faces a manual audit. With a design graph, the trace follows the outcome back through the items and objectives involved to the decisions behind them, and the reasons recorded with those decisions are there to read. In the opening scenario, the learning engineer would start from the biology cohort's metric and follow it back to the items students struggled with and the objective those items assess, then forward from that objective to the activities that target it and back to the need that produced it, reading at each step who made the link and why. The trace itself is built; bringing outcome data into the graph waits on the planned outcomes import, and the demonstration course has no outcome data yet.

AI becomes a design-aware, reviewable collaborator. A significant limitation of using AI in design is the cost of constructing the project-level context it needs; all six software architects in one study raised it (Zhou et al., 2026, pp. 25, 28). Armature is designed to build that context as a byproduct of regular work. With the graph as context, a generic request such as "write a quiz about epigenetics" becomes a specific one: write items that measure *this* objective at *this* level, target *this* known misconception, and avoid overlap with the items already in the bank. Because an AI agent writes through the same API as everyone else, its contributions carry its identity, follow the same rules, and can be reviewed, contested, and reversed like anyone else's.

Design practice becomes comparable. Once artifacts and relations are recorded in a common form, and the planned export profiles let graphs be compared across institutions, the questions posed in Section 2 become answerable. Comparison does not mean reducing design to algorithmic rules, but means that claims about which practices work can be tested against more than recollection.

Standards will be referenced rather than duplicated: the planned external references will let an objective point to a CASE competency and an item to its QTI representation, and the planned outcomes importer will join learning data to design data through shared identifiers.

5. Lessons from earlier attempts

Armature is not the first attempt to give design a durable record. This section is not a review of the literature involved. It is a check: what does the record of earlier attempts say about the architectural decisions Armature must make? Four bodies of work are reviewed, with the counter-cases that complicate the obvious reading of each.

Descriptive vs. executable learning design

IMS Learning Design (IMS LD), published in 2003, set out to describe whole units of learning in a notation formal enough that "automatic processing is possible" (Koper, 2001, p. 6), so that a design could be exchanged and run in any compliant player. The usual explanation for its lack of adoption was that it was too complex, though the one direct test of that explanation did not support it: after a 45-minute introduction, university teachers produced designs that matched an expert solution at about 76%, and the authors concluded that the barriers to adoption "appear to lie elsewhere" (Derntl et al., 2012, p. 74).

The specification's own experts named other barriers. At a 2008 workshop, thirteen IMS LD specialists reported that "it is not quite apparent to stakeholders, yet, what the core and key benefits of IMS LD are"; that runtime flexibility would "demand a robust definition of a package versioning system, in order to deal with change reverts, branches, multiple authors etc."; that information from deployment could not easily flow back into new releases; and that "the process followed and reasoning used by the teacher when creating the learning designs is not captured in the final design" (Neumann et al., 2010, pp. 60, 62–63, 69). Similar barriers were blamed when the executable sequencing layer of SCORM 2004 met the same fate. One practitioner observed that its benefit did not exceed its cost "even for the content developer who carries the substantially lighter burden" (Rustici, 2008, p. 3).

The counter-case matters. Visual design languages that executed nothing were also rarely used, and their developers blamed missing tools and the cost of learning the notation (Figl et al., 2010). Being descriptive is not, by itself, enough. Two lessons follow from these cases: the benefits of a design record must be apparent to the people asked to bear its cost, and the record must be able to change as designs do.

Design rationale systems

From the late 1980s, design rationale research built notations for recording the issues, options and arguments behind design decisions, including IBIS and its hypertext descendants, and Questions, Options and Criteria (MacLean et al., 1991). An extensive review found that the claim that explicit argument structure helps designers reason was "empirically much weaker than many assume", and that the overhead of these notations had been assumed rather than studied (Shum & Hammond, 1994, p. 27). Grudin (1994) identified the problem for collaborative software generally: such applications "often require additional work from individuals who do not perceive a direct benefit from the use of the application" (p. 97). For design rationale, the question became "who will bear the cost of producing design rationales for a particular artifact, and why?" (Lee, 1997, p. 83). Shipman and McCall (1999)

observed that the extra effort of formalizing "is not balanced by a payoff at the time the formalization is required and for the persons who must do the formalization" (p. 202).

Counter-cases are informative as well. Incremental formalization, designed around those lessons, saw little uptake in its own trials: "only a couple of suggestions were accepted" (Shipman & McCall, 1999, p. 220). Compendium, a descendant of IBIS, continued to be used where a trained facilitator built the maps live during meetings (Conklin et al., 2001, p. 6). According to its developers' own institution, DRed, another IBIS derivative, spread at Rolls-Royce once the company incorporated it into its engineering system (University of Cambridge, 2014). The lesson of the rationale literature is about who pays and when: records made outside the work, for someone else's later benefit, did not last; where rationale capture did last, the work itself, a facilitator, or an organizational process carried it.

Structure nobody uses

In business process modeling, less than 20% of the BPMN vocabulary is regularly used (Muehlen & Recker, 2008, p. 465). In the IEEE learning object metadata standard, implementers converged on a small subset, and the roles for instructional and media designers went essentially unrecorded (Friesen, 2004). In one repository, 46% of about 6,500 author-created metadata records were poor or unusable (Currier et al., 2004, p. 14).

Design data captured after the fact is also expensive. The Open University's retrospective mapping of module designs took specialists one to five days per module, depending on the account (Rienties & Toetenel, 2016; Toetenel & Rienties, 2016).

Theorists of instructional design languages point the same way: most design languages are "not formal, computable, or standard" (Gibbons & Brewer, 2005, p. 118), and they are "at the same time a tool for and a byproduct of designing" (Gibbons et al., 2008, p. 636). Structure that runs ahead of practice goes unused, structure added afterward is expensive, and the structure that works emerges from designing itself.

What lasted

The evidence for the practices that did last is also mixed. Architecture decision records, short files kept beside the code and superseded rather than deleted (Nygard, 2011), are a real but niche practice (and were adopted for the development of Armature and CoQui). Among 921 public repositories that use them, more than half hold five or fewer, and only 5% hold more than twenty (Buchgeher et al., 2023). Commit messages can carry rationale reliably where strong review norms require it. In the Linux kernel, a deliberately chosen best case, 98.9% of commits contain rationale (Dhaouadi et al., 2024, p. 419). Curriculum maps persist where accreditation requires that curricula be monitored and revised (Liaison Committee on Medical Education, 2026), though with substantial labor costs (Willett, 2008). And the US medical-device rules have required a design history file since 1997, though reasoning is not required (Food and Drug Administration, 1996). What these practices share is something outside the record that sustains it: review norms, an engineering process, accreditation, or regulation.

Lessons

Read together, these cases support five lessons. Armature's architecture is designed around the first four; the fifth is one it cannot meet on its own.

1. **Capture where the work happens.** Systems that asked people to record outside their work, in a separate tool or after the fact, went unused or were expensive.
2. **Do not demand structure before it is understood.** Formal vocabularies ahead of practice produced unused fields and resisted tools.
3. **Keep records revisable, and keep their history.** Designs change; records that could not be versioned, branched, and merged could not follow them.
4. **Benefit the person recording now.** Benefits that were deferred, or that accrued to someone else, did not sustain use.

5. **Something must sustain the record.** Every practice that lasted had a norm, a mandate or a facilitator behind it: kernel review, an engineering process, accreditation, a trained mapper.

How Armature answers the lessons

Each of Armature's main architectural decisions answers one of these lessons. Each also has evidence against it, and a risk that remains.

Capture where the work happens (lesson 1). Design data enters Armature through plugins that do something designers already need, and every change is a commit with its author and a required reason. This is the arrangement the design rationale literature recommended and rarely achieved: capture as a byproduct of a method people already use, so that "the user benefits from the method and the rationales are captured at a relatively low cost" (Lee, 1997, p. 82). CoQui is designed this way: a review facilitator must give a reason for every revision of an item. CoQui assists the facilitator by presenting the relevant discussion between designers and subject-matter experts, then turning that reason into the commit message on the review round's branch. Against it, Regli et al. (2000) warn that "providing a totally new application for storing and retrieving information increases overhead and correspondingly decreases the probability of a successful system introduction" (p. 231), and CoQui is a new application. Armature's bet is that CoQui is not a rationale tool: its primary purpose is to solve the bottleneck of expert item review on its own terms, and the record is what it leaves behind. This is the intention, though Armature's API is not yet mature enough to test this tool-hub connection.

Do not demand structure before it is understood (lesson 2). The schema is small. A relation becomes a record of its own only when it carries data, such as a prerequisite's type and rationale, and rationale stays free text until use shows what structure is warranted. A capability enters the hub only when at least two plausible plugins would need it, in the form they share: CoQui's review rounds became branches rather than a new record type, and its review workflow, which changed often during development, stayed inside CoQui. Against it, incremental formalization, the closest precedent, saw little uptake. The risk is that structure arrives too late for the analyses that depend on it.

Keep records revisable, and keep their history (lesson 3). Commits, branches, three-way merges, and reads at any past commit give a design record the lifecycle that IMS LD's own experts said their specification lacked (Neumann et al., 2010, pp. 62–63). Because history is never rewritten, Armature also differs from the rationalized documentation Parnas and Clements (1986) recommend, in which design documents are revised after each change "to look as if the change had been the original design" (p. 8). Against it, history's demonstrated value is modest: developers consult it constantly, but find a commit's author or colleagues a better source of deep rationale than the message itself (Codoban et al., 2015). A history's first use may be as a pointer to whom to ask. Armature also has no dedicated way yet to record the alternatives a designer considered and rejected.

Benefit the person recording now (lesson 4). Design intelligence is the answer: coverage, alignment, trace and impact return to the plugin while the designer works (Section 4). Other analyses are planned and more will be discovered. The evidence that structured relations pay off is real but narrow. In one lab study, a visualization of which concepts the chosen activities practised nearly eliminated one class of design error, practice on concepts not yet taught (Albó et al., 2022), and in an intelligent tutoring system a redesign based on a data-corrected mapping of items to skills improved post-test performance (d = .47) (Liu & Koedinger, 2017). Both used relations that researchers had curated. Indicators also invite gaming: "educators, possibly held to account with analytics-based performance indicators, may well come to design learning in a way that will enable them to evidence impact with these new tools" (Knight et al., 2014, p. 42). The benefit also has to reach the person who records. In CoQui, the facilitator writes the reasons and subject-matter experts write the findings; whether the record repays each of them is untested.

Describe; never execute. The executable layers of IMS LD and SCORM 2004 carried a runtime cost the field did not bear. Armature records intent, structure and rationale, and leaves execution to

delivery tools. Descriptive design languages faded too, which is why this decision depends on the four above.

Something must sustain the record (lesson 5). No architectural decision can create a norm, and instructional design has no equivalent of kernel review. What Armature can do is give a norm something to work with: item and placement statuses that a review must move, findings that cannot be dismissed without a reason, and branches on which a review round's changes wait until they are merged. CoQui is designed to put subject-matter expert review on these mechanisms. Accreditation and quality assurance could add an outside reason to keep the record, since they already require that curricula be monitored and revised (European Association for Quality Assurance in Higher Education (ENQA) et al., 2015; Liaison Committee on Medical Education, 2026), though mandated quality processes can become compliance exercises rather than improvement (Newton, 2000). A norm is also what turns required reasons into good ones: Armature can refuse a change without a reason, but it cannot make the reason worth reading (Pimentel et al., 2019).

6. Objections

Three objections deserve answers in their strongest form. Each is partly right, and each shapes the design.

Design is judgment, not recorded decision

The most serious objection comes from scholarship on design judgment. Judgment, in Nelson and Stolterman's account, "does not replicate decision making" (as cited in Lachheb & Boling (2021), p. 169). Designers make judgments continuously and mostly unconsciously, and the deepest of them are "not easy to access" (Lachheb & Boling, 2021, p. 172). Designers reason from cases, not from principles (Boling, 2010, p. 4), and Boling warns against extracting "lessons learned" from past designs and storing them as rules. A coverage view that turns alignment into a number is exactly the kind of complexity reduction that critics of evidence-based practice warn against (Biesta, 2010, p. 498).

Much of this is conceded. Armature does not capture judgment. A design history is raw material for the reflective design cases Boling calls for, not a design case itself. Making design practice comparable must not become extracting rules from it. And alignment pursued as compliance is a documented failure mode (Loughlin et al., 2021). Still, the judgment literature itself supports documentation. Lachheb and Boling (2021) advise designers to "document your design through documents and project management tools", so that "you will have an audit trail at the end of the project that helps you or anyone else to trace back what design decisions you have made" (p. 176). LEED demonstrated that when designers do record decisions, they include judgments: experience was cited as an influence in 53% of decisions and research in 13% (Totino & Kessler, 2025). Free-text reasons keep the record from forcing those judgments into a rational form. Capture at the moment of change matters, because students in one study found it hard to recall, later, the reasoning behind their earlier judgments (Demiral-Uzan & Boling, 2024, p. 1844). And under bounded rationality, notes from earlier projects serve as external memory that designers draw on when adapting past strategies (Stefaniak et al., 2023, p. 237).

The critique also shapes the architecture. Two responses are built: because the API refuses a write without a reason, every plugin must ask for one at the moment of change, when the reasoning is freshest; and design intelligence returns information with metrics instead of verdicts. Two more are proposed but not built: reasons that can be marked tentative, and an export of a module's history as raw material for a design case.

Every category carries values

Critics of classification argue that "each standard and each category valorizes some point of view and silences another" (Bowker & Star, 1999). Formalizing the nonformal is a source of technical bias (Friedman, 1996, p. 20). And methods of classification are "partially involved in the creation of the realities they claim to measure" (Perrotta & Williamson, 2018, p. 4).

Armature does not claim neutrality. Its schema puts the learning objective at the center, treats alignment between objectives, activities, and assessments as worth inspecting, and uses published taxonomies. Those are commitments, and they are declared as such, following the turn in learning design from neutrality to pluralism ("The Larnaca Declaration on Learning Design," 2016). What the design can offer critics is what Bowker and Star (1999) themselves recommend: keeping "the voices of classifiers and their constituents present", through authorship and reasons on every change, and leaving room for ambiguity, through free text. Relations stay revisable as goals change (Friedman, 1996, p. 19). If free text later points to a useful new taxonomy, the schema can be revised to include it, but never to the point of excluding free text options.

Records become surveillance

Reasons written down are accounts, shaped by an audience: developers who know they are watched make their commit messages "snappy" (Dabbish et al., 2012, p. 1285), and archived design conversations partly record a designer's "defense" of their decisions (Lachheb & Boling, 2021, p. 176). If the records are used to evaluate the people who write them, what is captured will be written for evaluators, and the candor that makes it worth keeping will evaporate. A field study of factory production lines found that making workers' activity observable can lead them to conceal it, and that "even a modest increase in group-level privacy sustainably and significantly improves line performance" (Bernstein, 2012, p. 181). Armature's decision records commit it never to aggregate records by person: no read computes a summary of one person's work, and a guard that refuses to list records by their author is planned. But the commitment cannot be enforced structurally, because anyone with bulk access to the history can compute what Armature declines to. So it needs a governance model. That model should say who holds a design graph, who may query it, how research exports are made, and how the commitment is protected against institutional pressure to evaluate staff. It should also account for labor and academic-freedom considerations where designers' records are concerned.

7. Conclusion

Learning engineering has learned to record what learners do. It has not yet learned to record how the designs they experience came to be. The result is the opening scenario, repeated across institutions: a struggling cohort, a question about why, and an answer scattered across documents and memory.

Armature proposes a system of record for design: artifacts, the named relations between them, and a history in which every change carries an author and a reason, filled by the plugins designers use and returning analysis while they work. Its architecture is designed to answer the reasons earlier attempts faded, it treats relations as the fallible, attributable claims the alignment literature shows them to be, and it declines to turn the record of a design into a record of the people who made it. What it cannot supply are reasons good enough to use and a norm that sustains the record; only use will show whether those follow. If design has a record, design practice becomes comparable across projects, AI contributions become reviewable, and "why did we do that?" has an answer.

Armature is an open project. Its schema, API and decision records are public under the MIT license at https://github.com/mossglenn/armature, and the author welcomes collaborators who want to improve the infrastructure, build a plugin, pilot it with a design team, or study the design data it records.

AI disclosure

The author used Claude (Anthropic) throughout this work. In developing Armature, Claude served as a sounding board, proposed schema and code, and drafted decision records and documentation. All conceptualization, framing, and design intent originated with the author, and every commit was reviewed and approved by the author. For this paper, Claude assisted in searching the literature, extracted quotations, proposed structure, and drafted text. All language, arguments and claims were revised by the author and reflect the author's own judgment.

References

1EdTech Consortium. (n.d.). *Competencies and Academic Standards Exchange (CASE) Service Version 1.1 Implementation Guide*. Retrieved https://www.imsglobal.org/spec/CASE/v1p1/impl

Albó, L., Barria-Pineda, J., Brusilovsky, P., & Hernández-Leo, D. (2022). Knowledge-Based Design Analytics for Authoring Courses with Smart Learning Content. *International Journal of Artificial Intelligence in Education*, *32*(1), 4–27. https://doi.org/10.1007/s40593-021-00253-3

Aleven, V., McLaughlin, E. A., Glenn, R. A., & Koedinger, K. R. (2026). Instruction based on adaptive learning technologies. In R. E. Mayer, P. A. Alexander, & L. Fiorella (Eds.), *Handbook of research on learning and instruction* (pp. 477–518). Routledge. https://doi.org/10.4324/9781003484776

Anderson, L. W., & Krathwohl, D. R. (Eds.). (2001). *A taxonomy for learning, teaching, and assessing: A revision of Bloom's taxonomy of educational objectives* (Complete ed.). Longman.

Bernstein, E. S. (2012). The Transparency Paradox: A Role for Privacy in Organizational Learning and Operational Control. *Administrative Science Quarterly*, *57*(2), 181–216. https://doi.org/10.1177/0001839212453028

Biesta, G. J. J. (2010). Why "What Works" Still Won't Work: From Evidence-Based Education to Value-Based Education. *Studies in Philosophy and Education*, *29*(5), 491–503. https://doi.org/10.1007/s11217-010-9191-x

Biggs, J. (1996). Enhancing teaching through constructive alignment. *Higher Education*, *32*(3), 347–364. https://doi.org/10.1007/BF00138871

Boling, E. (2010). The Need for Design Cases: Disseminating Design Knowledge. *International Journal of Designs for Learning*, *1*(1). https://doi.org/10.14434/ijdl.v1i1.919

Bowker, G. C., & Star, S. L. (1999). *Sorting Things Out: Classification and Its Consequences*. The MIT Press. https://doi.org/10.7551/mitpress/6352.001.0001

Buchgeher, G., Schöberl, S., Geist, V., Dorninger, B., Haindl, P., & Weinreich, R. (2023). Using Architecture Decision Records in Open Source Projects—An MSR Study on GitHub. *IEEE Access*, *11*, 63725–63740. https://doi.org/10.1109/ACCESS.2023.3287654

CaSS Project. (n.d.). *Assertion processing*. CaSS Docs (v1.6). Retrieved October 15, 2026, from https://docs.cassproject.org/v1.6/guide/assertion-processing/#requirements

Cen, H., Koedinger, K., & Junker, B. (2006). *Learning Factors Analysis – A General Method for Cognitive Model Evaluation and Improvement* (pp. 164–175). Springer Berlin Heidelberg. https://doi.org/10.1007/11774303_17

Codoban, M., Ragavan, S. S., Dig, D., & Bailey, B. (2015). Software history under the lens: A study on why and how developers examine it. *2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 1–10. https://doi.org/10.1109/ICSM.2015.7332446

Conklin, J., Selvin, A., Shum, S. B., & Sierhuis, M. (2001). *Facilitated hypertext for collective sensemaking: 15 years on from gIBIS*. https://doi.org/10.1145/504243.504246

Currier, S., Barton, J., O'Beirne, R., & Ryan, B. (2004). Quality assurance for digital learning object repositories: Issues for the metadata creation process. *Research in Learning Technology*, *12*(1). https://doi.org/10.3402/rlt.v12i1.11223

Czerwinski, E., Goodell, J., Ritter, S., Sottilare, R., Thai, K.-P., & Jacobs, D. (2023). Learning engineering uses data (part 1): Instrumentation. In J. Goodell & J. Kolodner (Eds.), *Learning Engineering Toolkit: Evidence-Based Practices from the Learning Sciences, Instructional Design, and Beyond* (1st ed., pp. 153–174). Routledge. https://doi.org/10.4324/9781003276579-9

Dabbish, L., Stuart, C., Tsay, J., & Herbsleb, J. (2012). Social coding in GitHub: Transparency and collaboration in an open software repository. *Proceedings of the ACM 2012 Conference on Computer Supported Cooperative Work*, 1277–1286. https://doi.org/10.1145/2145204.2145396

Demiral-Uzan, M., & Boling, E. (2024). Instructional design students' design judgment development. *Educational Technology Research and Development*, *72*(3), 1813–1849. https://doi.org/10.1007/s11423-024-10361-1

Derntl, M., Neumann, S., Griffiths, D., & Oberhuemer, P. (2012). The Conceptual Structure of IMS Learning Design Does Not Impede Its Use for Authoring. *IEEE Transactions on Learning Technologies*, *5*(1), 74–86. https://doi.org/10.1109/TLT.2011.25

Dhaouadi, M., Oakes, B. J., & Famelis, M. (2024). *Rationale Dataset and Analysis for the Commit Messages of the Linux Kernel Out-of-Memory Killer*. 415–425. https://doi.org/10.1145/3643916.3644413

European Association for Quality Assurance in Higher Education (ENQA), European Students' Union (ESU), European University Association (EUA), & European Association of Institutions in Higher Education (EURASHE). (2015). *Standards and guidelines for quality assurance in the European Higher Education Area (ESG)*. EURASHE. https://www.enqa.eu/wp-content/uploads/2015/11/ESG_2015.pdf

Figl, K., Derntl, M., Rodriguez, M. C., & Botturi, L. (2010). Cognitive effectiveness of visual instructional design languages. *Journal of Visual Languages & Computing*, *21*(6), 359–373. https://www.sciencedirect.com/science/article/pii/S1045926X10000509

Food and Drug Administration. (1996). Medical devices; Current good manufacturing practice (CGMP) final rule; Quality system regulation. *Federal Register*, *61*(195), 52602–52662. https://www.govinfo.gov/content/pkg/FR-1996-10-07/html/96-25720.htm

Friedman, B. (1996). Value-sensitive design. *Interactions*, *3*(6), 16–23. https://doi.org/10.1145/242485.242493

Friesen, N. (2004). The International Learning Object Metadata Survey. *The International Review of Research in Open and Distributed Learning*, *5*(3). https://doi.org/10.19173/irrodl.v5i3.195

Gibbons, A. S., Botturi, L., Boot, E., & Nelson, J. (2008). Design languages. In *Handbook of research on educational communications and technology* (pp. 633–645). Routledge. https://www.researchgate.net/profile/Luca-Botturi/publication/268414155_47_Design_Languages/links/54f060040cf2432ba65aa12c/47-Design-Languages.pdf

Gibbons, A. S., & Brewer, E. K. (2005). Elementary principles of design languages and design notation systems for instructional design. In J. M. Spector, C. Ohrazda, A. Van Schaack, & D. A. Wiley (Eds.), *Innovations in Instructional Technology: Essays in Honor of M. David Merrill* (pp. 111–129). Lawrence Erlbaum Associates. https://doi.org/10.4324/9781410613684-16

Groth, P., Gibson, A., & Velterop, J. (2010). The anatomy of a nanopublication. *Information Services and Use*, *30*(1–2), 51–56. https://doi.org/10.3233/ISU-2010-0613

Grudin, J. (1994). Groupware and social dynamics: Eight challenges for developers. *Communications of the ACM*, *37*(1), 92–105. https://doi.org/10.1145/175222.175230

Harden, R. M. (2001). AMEE Guide No. 21: Curriculum mapping: A tool for transparent and authentic teaching and learning. *Medical Teacher*, *23*(2), 123–137. https://doi.org/10.1080/01421590120036547

Hayashi, Y., Bourdeau, J., & Mizoguchi, R. (2009). Using ontological engineering to organize learning/instructional theories and build a theory-aware authoring system. *International Journal of Artificial Intelligence in Education*, *19*(2), 211–252. https://doi.org/10.3233/IRG-2009-19(2)05

Herman, J. L., Webb, N. M., & Zuniga, S. A. (2005). *Measurement issues in the alignment of standards and assessments: A case study* [CSE Report]. National Center for Research on Evaluation, Standards, and

Student Testing (CRESST), University of California, Los Angeles. https://files.eric.ed.gov/fulltext/ED488723.pdf

Hernández-Leo, D., Asensio-Pérez, J. I., Derntl, M., Pozzi, F., Chacón, J., Prieto, L. P., & Persico, D. (2018). An integrated environment for learning design. *Frontiers in ICT*, *5*, 9. https://doi.org/10.3389/fict.2018.00009

Hernández-Leo, D., Martinez-Maldonado, R., Pardo, A., Muñoz-Cristóbal, J. A., & Rodríguez-Triana, M. J. (2019). Analytics for learning design: A layered framework and tools. *British Journal of Educational Technology*, *50*(1), 139–152. https://doi.org/10.1111/bjet.12645

IEEE ICICLE. (n.d.). *Learning Engineering Process*. Retrieved October 15, 2026, from https://sagroups.ieee.org/icicle/learning-engineering-process

Jonassen, D. H. (2008). Instructional Design as Design Problem Solving: An Iterative Process. *Educational Technology*, *48*(3), 21–26.

Kaufman, R. A. (1972). *Educational system planning*. Prentice-Hall.

Kessler, A., Craig, S. D., Goodell, J., Kurzweil, D., & Greenwald, S. W. (2023). Learning engineering is a process. In J. Goodell & J. Kolodner (Eds.), *Learning Engineering Toolkit: Evidence-Based Practices from the Learning Sciences, Instructional Design, and Beyond* (1st ed., pp. 29–45). Routledge. https://doi.org/10.4324/9781003276579-5

Knight, S., Buckingham Shum, S., & Littleton, K. (2014). Epistemology, Assessment, Pedagogy: Where Learning Meets Analytics in the Middle Space. *Journal of Learning Analytics*, *1*(2). https://doi.org/10.18608/jla.2014.12.3

Koedinger, K. R., Baker, R. S. J. d., Cunningham, K., Skogsholm, A., Leber, B., & Stamper, J. (2010). A data repository for the EDM community: The PSLC DataShop. In C. Romero, S. Ventura, M. Pechenizkiy, & R. S. J. d. Baker (Eds.), *Handbook of educational data mining* (pp. 43–56). CRC Press. https://doi.org/10.1201/b10274-10

Koper, R. (2001). *Modeling units of study from a pedagogical perspective: The pedagogical meta-model behind EML*. https://research.ou.nl/files/1045171/Pedagogical%20metamodel%20behind%20EMLv2.pdf

Lachheb, A., & Boling, E. (2021). The Role Of Design Judgment And Reflection In Instructional Design. In J. K. McDonald & R. E. West (Eds.), *Design for Learning: Principles, Processes, and Praxis* (pp. 169–179). EdTech Books. https://edtechbooks.org/id/design_judgment

Laurillard, D., Kennedy, E., Charlton, P., Wild, J., & Dimakopoulos, D. (2018). Using technology to develop teachers as designers of TEL: Evaluating the Learning Designer. *British Journal of Educational Technology*, *49*(6), 1044–1058. https://doi.org/10.1111/bjet.12697

Lee, J. (1997). Design rationale systems: Understanding the issues. *IEEE Expert*, *12*(3), 78–85. https://doi.org/10.1109/64.592267

Liaison Committee on Medical Education. (2026). *Functions and structure of a medical school: Standards for accreditation of medical education programs leading to the MD degree*. Liaison Committee on Medical Education. https://lcme.org/wp-content/uploads/2026/03/2027-28-Functions-and-Structure_2026-03-13.docx

Liu, R., & Koedinger, K. R. (2017). Closing the loop: Automated data-driven cognitive model discoveries lead to improved instruction and learning gains. *Journal of Educational Data Mining*, *9*(1). https://jedm.educationaldatamining.org/index.php/JEDM/article/view/212

Loughlin, C., Lygo-Baker, S., & Lindberg-Sand, Å. (2021). Reclaiming constructive alignment. *European Journal of Higher Education*, *11*(2), 119–136. https://doi.org/10.1080/21568235.2020.1816197

MacLean, A., Young, R., Bellotti, V., & Moran, T. (1991). Questions, Options, and Criteria: Elements of Design Space Analysis. *Human-Computer Interaction*, *6*(3–4), 201–250. https://doi.org/10.1207/s15327051hci0603&4_2

Mangaroska, K., & Giannakos, M. (2019). Learning Analytics for Learning Design: A Systematic Literature Review of Analytics-Driven Design to Enhance Learning. *IEEE Transactions on Learning*

Technologies, *12*(4), 516–534. IEEE Transactions on Learning Technologies. https://doi.org/10.1109/TLT.2018.2868673

Moore, S., Schmucker, R., Mitchell, T., & Stamper, J. (2024). Automated Generation and Tagging of Knowledge Components from Multiple-Choice Questions. *Proceedings of the Eleventh ACM Conference on Learning @ Scale*, 122–133. https://doi.org/10.1145/3657604.3662030

Muehlen, M. Z., & Recker, J. (2008). *How Much Language Is Enough? Theoretical and Practical Use of the Business Process Modeling Notation* (pp. 465–479). Springer International Publishing. https://doi.org/10.1007/978-3-540-69534-9_35

Muşlu, K., Bird, C., Nagappan, N., & Czerwonka, J. (2014). Transition from centralized to decentralized version control systems: A case study on reasons, barriers, and outcomes. *Proceedings of the 36th International Conference on Software Engineering*, 334–344. https://doi.org/10.1145/2568225.2568284

Neumann, S., Klebl, M., Griffiths, D., Hernández-Leo, D., de la Fuente Valentín, L., Hummel, H., Brouns, F., Derntl, M., & Oberhuemer, P. (2010). Report of the Results of an IMS Learning Design Expert Workshop. *International Journal of Emerging Technologies in Learning (iJET)*, *5*(1), 58–72. https://doi.org/10.3991/ijet.v5i1.1045

Newton, J. (2000). Feeding the beast or improving quality? Academics' perceptions of quality assurance and quality monitoring. *Quality in Higher Education*, *6*(2), 153–163. https://doi.org/10.1080/713692740

Nygard, M. (2011, November 15). *Documenting architecture decisions*. Cognitect Blog. https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions

Open edX. (2026). *Openedx-core: Openedx_content architecture decision records*. https://github.com/openedx/openedx-core/tree/main/docs/openedx_content/decisions

Parnas, D. L., & Clements, P. C. (1986). A rational design process: How and why to fake it. *IEEE Transactions on Software Engineering*, *SE-12*(2), 251–257. https://doi.org/10.1109/TSE.1986.6312940

Perrotta, C., & Williamson, B. (2018). The social life of Learning Analytics: Cluster analysis and the "performance" of algorithmic education. *Learning, Media and Technology*, *43*(1), 3–16. https://doi.org/10.1080/17439884.2016.1182927

Pimentel, J. F., Freire, J., Murta, L., & Braganholo, V. (2019). A survey on collecting, managing, and analyzing provenance from scripts. *ACM Computing Surveys*, *52*(3), 1–38. https://doi.org/10.1145/3311955

Regli, W. C., Hu, X., Atwood, M., & Sun, W. (2000). A Survey of Design Rationale Systems: Approaches, Representation, Capture and Retrieval. *Engineering with Computers*, *16*(3–4), 209–235. https://doi.org/10.1007/PL00013715

Rienties, B., & Toetenel, L. (2016). The impact of learning design on student behaviour, satisfaction and performance: A cross-institutional comparison across 151 modules. *Computers in Human Behavior*, *60*, 333–341. https://doi.org/10.1016/j.chb.2016.02.074

Robson, R., Goldberg, B., Blake-Plock, S., Casey, C., Hoyt, W., Hernandez, M., & Ray, F. (2022). Mining Artificially Generated Data to Estimate Competency. In A. Mitrovic & N. Bosch (Eds.), *Proceedings of the 15th International Conference on Educational Data Mining* (pp. 828–833). International Educational Data Mining Society. https://doi.org/10.5281/zenodo.6852926

Rustici, M. (2008). *SCORM: The baby and the bathwater*. https://scorm.com/wp-content/assets/LETSI-white-papers/LETSI%20White%20Papers%20-%20Rustici%20-%20SCORM_The_Baby_and_the_Bathwater.pdf

Samuelsen, J., Chen, W., & Wasson, B. (2021). Enriching context descriptions for enhanced LA scalability: A case study. *Research and Practice in Technology Enhanced Learning*, *16*(1). https://doi.org/10.1186/s41039-021-00150-2

Sandoval, W. (2014). Conjecture Mapping: An Approach to Systematic Educational Design Research. *Journal of the Learning Sciences*, *23*(1), 18–36. https://doi.org/10.1080/10508406.2013.778204

Schatz, S., Thai, K.-P., Craig, S. D., Schoenherr, J. R., Lis, J., & Kolodner, J. (2023). Human-centered design tools. In J. Goodell & J. Kolodner (Eds.), *Learning Engineering Toolkit: Evidence-Based Practices from the Learning Sciences, Instructional Design, and Beyond* (1st ed., pp. 279–301). Routledge. https://doi.org/10.4324/9781003276579-17

Shipman, F. M., & McCall, R. J. (1999). Incremental formalization with the hyper-object substrate. *ACM Transactions on Information Systems*, *17*(2), 199–227. https://doi.org/10.1145/306686.306690

Shum, S. B., & Hammond, N. (1994). Argumentation-based design rationale: What use at what cost? *International Journal of Human-Computer Studies*, *40*(4), 603–652. https://doi.org/10.1006/ijhc.1994.1029

Simon Initiative, Carnegie Mellon University. (2026). *OLI Torus design documents*. https://github.com/Simon-Initiative/oli-torus/tree/master/docs/design-docs

Stamper, J. C., & Koedinger, K. R. (2011). *Human-Machine Student Model Discovery and Improvement Using DataShop* (pp. 353–360). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-21869-9_46

Star, S. L., & Ruhleder, K. (1996). Steps toward an ecology of infrastructure: Design and access for large information spaces. *Information Systems Research*, *7*(1), 111–134. https://doi.org/10.1287/isre.7.1.111

Stefaniak, J., Tawfik, A., & Sentz, J. (2023). Supporting Dynamic Instructional Design Decisions Within a Bounded Rationality. *TechTrends*, *67*(2), 231–244. https://doi.org/10.1007/s11528-022-00792-z

Thai, K.-P., Craig, S. D., Goodell, J., Lis, J., Schoenherr, J. R., & Kolodner, J. (2023). Learning engineering is human-centered. In J. Goodell & J. Kolodner (Eds.), *Learning Engineering Toolkit: Evidence-Based Practices from the Learning Sciences, Instructional Design, and Beyond* (pp. 83–123). Routledge. https://doi.org/10.4324/9781003276579-7

The Larnaca Declaration on Learning Design. (2016). *Journal of Interactive Media in Education*, *2016*(1). https://doi.org/10.5334/jime.407

Tian, Y., Zhang, Y., Stol, K.-J., Jiang, L., & Liu, H. (2022). What makes a good commit message? *Proceedings of the 44th International Conference on Software Engineering*, 2389–2401. https://doi.org/10.1145/3510003.3510205

Toetenel, L., & Rienties, B. (2016). Analysing 157 learning designs using learning analytic approaches as a means to evaluate the impact of pedagogical decision making. *British Journal of Educational Technology*, *47*(5), 981–992. https://doi.org/10.1111/bjet.12423

Totino, L., & Kessler, A. (2024). "Why did we do that?" A Systematic Approach to Tracking Decisions in the Design and Iteration of Learning Experiences. *The Journal of Applied Instructional Design*. https://doi.org/10.59668/1269.15630

Totino, L., & Kessler, A. (2025). Exploring Learning Engineering Design Decision Tracking: Emergent Themes from Practitioners' Work. *TechTrends*. https://doi.org/10.1007/s11528-025-01076-y

University of Cambridge. (2014). *DRed* [REF2014 impact case study]. University of Cambridge. https://impact.ref.ac.uk/casestudies/CaseStudy.aspx?Id=14057

Webb, N. L. (2007). Issues Related to Judging the Alignment of Curriculum Standards and Assessments. *Applied Measurement in Education*, *20*(1), 7–25. https://doi.org/10.1080/08957340709336728

Wiggins, G., & McTighe, J. (2005). *Understanding by design* (Expanded 2nd ed.). Association for Supervision and Curriculum Development.

Willett, T. G. (2008). Current status of curriculum mapping in Canada and the UK. *Medical Education*, *42*(8), 786–793. https://doi.org/10.1111/j.1365-2923.2008.03093.x

Zhou, X., Li, R., Liang, P., Zhang, B., Shahin, M., Li, Z., & Yang, C. (2026). Using LLMs in Generating Design Rationale for Software Architecture Decisions. *ACM Transactions on Software Engineering and Methodology*, *35*(8), 1–38. https://doi.org/10.1145/3785010